Do Now · retrieval
Sequence & branching in C#

Name: _____ Class: _____ Date: _____

Answer four retrieval questions before reviewing earlier learning.

Q1

Why would Unicode be chosen instead of standard ASCII for a multilingual application?

Q2

Convert 01011000 0011 to denary using an 8-bit two’s-complement mantissa and a 4-bit two’s-complement exponent.

Q3

Convert $6D_{16}$ to eight-bit binary.

Q4

Convert 11110110 from 8-bit two’s-complement binary to denary. Show signed place values.

New Learning · key ideas

The important knowledge from today's lesson. Read, annotate, and explain it.

Core idea 1

Sequence = instructions in order — Sequence is the normal top-to-bottom execution of statements. Each statement happens after the previous one unless another construct changes the flow. — CODE:

```
Console.Write("Name: ");
string name = Console.ReadLine();
Console.WriteLine("Hello " + name);
```

Core idea 2

Input arrives as text — Console.ReadLine() returns a string. If the program needs a whole number, parse the input into an int . — CODE:

```
Console.Write("Age: ");
int age = int.Parse(Console.ReadLine());
```

Core idea 3

Branching chooses a route — An if condition must evaluate to true or false . Only the matching branch is executed. — CODE:

```
if (age >= 18)
{
Console.WriteLine("Adult");
}
else
{
Console.WriteLine("Under 18");
}
```

New Learning · applying the ideas

Core idea 4

Conditions and order matter — Use == to compare equality, not = . Use && , || and ! for AND, OR and NOT. — With else if , conditions are tested from top to bottom. Put the most demanding threshold first.

Vocabulary · use the correct technical terms

Sequence — Instructions executed in their stated order.

Branching / selection — Choosing a route through a program according to a condition.

Boolean expression — An expression that evaluates to true or false.

Condition — The test inside an if statement.

Independent skill check · apply the lesson

Write C# if / else if / else to label ages 0-11 Child, 12-17 Teen, 18+ Adult. Test 11, 12, 17 and 18.

More animated examples are available in the original online lesson.

I Do · worked example

Problem

Build a result checker

Follow the worked method

1. Use the standard .NET Fiddle structure — CODE:
using System;

public class Program
{
public static void Main()
{
// code goes here
}
} — Main program instructions belong inside Main() .

2. Test the highest threshold first — CODE:
if (score >= 70)
{
Console.WriteLine("Distinction");
}
else if (score >= 50)
{
Console.WriteLine("Pass");
}
else
{
Console.WriteLine("Not yet");
} — A score of 85 satisfies both >= 70 and >= 50 , so the higher threshold must be tested first.

3. Use values that expose mistakes — CODE:
85 → Distinction
70 → Distinction
50 → Pass
49 → Not yet

Why this method works

Identify the decision, rule or invariant that makes the method correct.

We Do · Try → Discuss → Improve

TRY independently in BLACK PEN; compare methods with a partner; improve your reasoning.

Guided problem · TRY

Build the missing decision

CODE:

```
Console.Write("Number: ");
int number = int.Parse(Console.ReadLine());

// ?
```

DISCUSS

Compare how you reached the answer. Where could someone make an error?

IMPROVE

Add one precise correction or a stronger reason. Answers are not printed here.

You Do · independent practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 1

Positive, negative or zero
Ask the user for an integer. Output exactly one of:
Positive · Negative · Zero

Check that you have used the technical terms and explained your decisions.

You Do · extended practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 2

Delivery charge
Ask for an order total and whether the customer is a member.
Output Free delivery only when the total is at least 30 AND the member answer is "yes" .
Otherwise output Delivery charge applies .

Check that you have used the technical terms and explained your decisions.

Exit Ticket · reflect and retrieve

Exit Ticket

In C#, == is used to compare two values for equality.

Use key vocabulary from memory

1. Explain Sequence in your own words.

2. Explain Branching / selection in your own words.

3. Explain Boolean expression in your own words.

Confidence and next step

How confident are you? Thumbs up / side / down

Which question would you improve after checking the classroom model?

Check answers after the lesson using the teacher-guided source models.